ASIF SAAS STUDIO

QR Code Generator

Complete documentation: a detailed how-to-use guide for all eight content types, field and settings references, the hand-written QR engine explained, validation rules, deployment and testing for the browser-only QR code application.

Creator: Dr. Asif Ahmad
Version: 1.0 · **Category:** Design
Date: 20 October 2026
Deliverable: a single self-contained `index.html` (41.6 KB) — no server, no build step, no accounts, no uploads.

1 · Introduction

QR Code Generator creates real, scannable QR codes entirely inside the visitor's browser. The user first decides *what kind of information* the code should carry — a website link, Wi-Fi credentials, a contact card, an email, an SMS, a phone number, a location, or plain text — then fills in exactly the fields for that kind, and downloads the finished code as a PNG image or an SVG file.

The application was built for the Asif SaaS Studio tools catalogue, where tools run inside a sandboxed iframe that forbids API keys, browser storage, service workers and permission prompts. There is no QR library and no web service behind it: the QR encoder — Reed–Solomon error correction, block interleaving, mask evaluation, matrix construction — is written from the ISO/IEC 18004 specification directly into the page. The codes it produces are standard QR Code Model 2 symbols (versions 1–20) that any phone camera or scanner app can read.

Typical uses

- A guest Wi-Fi code on a café table, office reception or hospital waiting area — guests join without asking for the password.
- A digital visiting card: one scan saves your name, phone, email and organisation into the phone's contacts.
- Posters and handouts linking to a website, registration form, video or menu.
- Product labels and classroom materials with pre-written SMS or email codes, or a map location.

> **Why local generation matters.** Many free QR websites quietly wrap your link in their own redirect (so they can track scans or start charging later — and the code dies if they shut down). This generator encodes *your content directly*: the code you download today works forever, and nothing you type is ever transmitted anywhere.

2 · Features at a glance

Feature	What it does
8 content types	Text, Website, Wi-Fi, Contact card (vCard), Email, SMS, Phone, Location — you decide which fields go into the code.
Type-specific fields	Each type shows its own form with the right fields, hints and required-field marks.
Live content preview	The exact text that will be encoded — and its byte size — updates as you type.
Full validation	Required fields, email and phone formats, URL parsing, coordinate ranges and length limits, with the error pinned to the exact field.
Error-correction levels	L (7%), M (15%), Q (25%) and H (30%) — higher levels survive damage, dirt and partial covering.
PNG + SVG downloads	PNG at 256–2048 px for screens and print; SVG for infinitely sharp printing at any size.
Standards-true payloads	Wi-Fi `WIFI:` format with spec escaping, vCard 3.0 contact cards, `mailto:`, `SMSTO:`, `tel:` and `geo:` formats phones understand natively.
Hand-written QR engine	ISO 18004 byte-mode encoder: smallest fitting version chosen automatically (1–20), Reed–Solomon error correction, best-mask selection by the spec's penalty rules.

3 · How to use it — step by step

Step 1 — Open the tool

Open `index.html` in any modern browser, or open *QR Code Generator* from the Asif SaaS Studio catalogue. Nothing to install, no account. The screen has two panels: **“1 · Choose what the code contains”** and **“2 · Your QR code”** (stacked on a phone).

Step 2 — Choose the content type

Tap one of the eight type buttons at the top: 📝 **Text** · 🔗 **Website** · 📶 **Wi-Fi** · 👤 **Contact** · ✉️ **Email** · 💬 **SMS** · 📞 **Phone** · 📍 **Location**. The form below changes to show exactly the fields for that type. Switching types never erases what you typed in other types.

Step 3 — Fill in the fields

Fields marked with a red ***** are required; the rest are optional and are simply left out of the code when empty. Per type:

- **Text:** type or paste anything — a message, a serial number, a poem. What you type is exactly what the scanner will show.
- **Website:** type the address in any of these forms: `example.com`, `www.example.com/page`, `https://example.com`. If the `https://` part is missing it is added for you.
- **Wi-Fi:** enter the network name (SSID) exactly as it appears in Wi-Fi settings (capital letters matter), pick the security type (most home/office networks are WPA/WPA2), type the password, and tick *Hidden network* only if the network does not broadcast its name. Choose *No password* for open networks — the password box then switches off.
- **Contact:** full name is required. Add any of organisation, job title, phone, email, website, postal address parts and a note. The scanner's phone will offer to save all of it as one contact. Write the phone in international format (e.g. `+92 300 1234567`) so it works from any country.
- **Email:** recipient address (required), plus optional subject and message — scanning opens the visitor's mail app with everything pre-filled.
- **SMS:** a phone number (required) and an optional pre-written message.
- **Phone:** just the number — scanning offers to call it.
- **Location:** latitude and longitude as decimal numbers (e.g. `33.7294` and `73.0931`). In Google Maps, long-press (phone) or right-click (computer) any place to see its coordinates.

> **Watch the “Encoded content” box.** On the right it shows, live, the exact text the code will carry and how many bytes it is. It is the ground truth — if something looks wrong there, fix the field before generating.

Step 4 — Choose error correction

Error correction lets a code survive damage. **M (15%)** is the right default for screens and clean print. Choose **Q** or **H** when the printed code will be outdoors, handled a lot, or partly covered by a logo or sticker — at the price of a denser pattern and less room for content. Choose **L** when the content is long and the code will be displayed on a screen.

Step 5 — Generate

Tap ⬛ **Generate QR Code**. If a field is invalid, generation stops: the field is outlined in red with the reason written under it, and a summary appears. Fix and tap again. On success the code appears in the preview panel, with a line underneath reporting its **version** (pattern size), module count, error-correction level and the PNG pixel size.

Step 6 — Choose a size and download

- **PNG size:** 256 px for small web use, 512 px (default) for documents and screens, 1024/2048 px for print. Changing the size re-creates the PNG immediately.
- ⬇ **Download PNG** — a normal image file; use it in Word, PowerPoint, Canva, or print it directly.
- ⬇ **Download SVG** — a vector file that stays perfectly sharp at any size; ideal for professional printing, banners and packaging.

Step 7 — Test before you share

Point your own phone camera at the *printed or displayed* code and confirm it does what you expect (opens the link, joins the Wi-Fi, saves the contact…). Test at the real size and distance — a dense code that scans from 10 cm on screen may fail across a room on a poster.

Worked example — a guest Wi-Fi code

Choose **Wi-Fi** → SSID `Office-Guest` → Security *WPA/WPA2/WPA3* → password typed → leave Hidden unticked → Error correction **Q** (it will be printed and handled) → **Generate** → PNG size *1024* → **Download PNG** → print and frame it for the reception desk. Guests scan, tap “join”, and are online — no one needs to spell the password aloud again.

> **A QR code is not a secret.** Anyone who can see a Wi-Fi code can read its password (scanning apps show the password in plain text). That is the point of a guest code — but use a guest network, not your main one, for codes displayed in public.

4 · Content types & fields reference

Type	Fields (* = required)	Encoded format
Text	Text *	The text itself, encoded as UTF-8.
Website	Address *	The URL as typed, with `https://` added when no scheme is present.
Wi-Fi	SSID *, Security, Password *, Hidden	`WIFI:T:WPA;S:name;P:password;;` — special characters (`\ ; , " :`) are backslash-escaped; `H:true;` added for hidden networks; password omitted for open networks.
Contact	Full name *, Org, Title, Phone, Email, Website, Street, City, Postal code, Country, Note	vCard 3.0 (`BEGIN:VCARD … END:VCARD`) with `N` / `FN` , `ORG` , `TITLE` , `TEL;TYPE=CELL` , `EMAIL` , `URL` , `ADR` , `NOTE` ; commas/semicolons/newlines escaped per the vCard rules.
Email	To *, Subject, Message	`mailto:to?subject=…&body=…` with percent-encoding; empty parameters omitted.
SMS	Number *, Message	`SMSTO:number:message` (message part omitted when empty).
Phone	Number *	`tel:number` — spaces and brackets are removed, a leading `+` is kept.
Location	Latitude *, Longitude *	`geo:lat,lng` in decimal degrees.

5 · Settings reference

Setting	Options	Default	Notes
Error correction	L · M · Q · H	M	Restores ≈7% / 15% / 25% / 30% of damaged codewords. Maximum content at the largest version (20): about 858 bytes at L, 666 at M, 482 at Q, 382 at H — the app computes the true limit for the chosen level and reports it in its error message.
PNG size	256 · 512 · 1024 · 2048 px	512	The PNG is rendered on whole-module boundaries, so the actual pixel size may be a little under the selected value (shown in the result line) — this keeps every module perfectly square and sharp.
SVG	—	—	Vector version of the same code, drawn at 8 px per module with a standard 4-module quiet zone; scales to any print size.

Capacity note: a QR symbol's capacity depends on its version, which the app chooses automatically as the smallest that fits. The figures above are the ceilings at the largest supported version (20); most real content — links, Wi-Fi credentials, full contact cards — lands between versions 2 and 10.

6 · Input validation

Check	Rule and behaviour
Required fields	Empty required fields are flagged when Generate is tapped; nothing is generated until they are fixed.
Website	Must parse as an http/https address after the automatic `https://` prefix; otherwise an error names the field.
Email format	Must match a standard address shape (`name@domain.tld`) — checked for the Email type and the contact card's email.
Phone format	5–24 characters of digits, spaces, brackets, dashes and an optional leading `+` — checked for SMS, Phone and contact card.
Coordinates	Latitude must be a number from −90 to 90; longitude from −180 to 180.
Wi-Fi password	Required unless Security is “No password”.
Length	If the content exceeds the capacity of a version-20 symbol at the chosen error level, a message states the approximate limit and suggests shortening the content or lowering the error level.

7 · How it works — the QR engine

The encoder implements QR Code Model 2 (ISO/IEC 18004) in byte mode, in six stages:

1. **Payload building.** The field values are assembled into the type's standard payload string (Section 4), then converted to UTF-8 bytes — so Arabic, Urdu and emoji text encode correctly.

2. **Version selection & bit stream.** The smallest version (1–20) whose data capacity fits is chosen. The stream is: mode indicator (`0100`), character count (8 bits up to version 9, 16 bits from version 10), the data bytes, a terminator, alignment to a byte boundary, and alternating pad bytes `EC 11` to exactly fill the data capacity.

3. **Error correction.** The data is split into the version's blocks (one or two groups, per the specification's table). For each block, Reed–Solomon parity is computed over GF(256) with the primitive polynomial `0x11D` using a generated divisor polynomial; data and parity codewords are then interleaved column-wise — which is what lets the code survive localised damage.

4. **Matrix construction.** Finder patterns with separators, timing patterns, alignment patterns (positions per version), the dark module, and reserved format/version areas are drawn; version information (BCH-coded) is added from version 7 up. The codeword bits are placed in the standard two-column zigzag from the bottom-right, skipping column 6.

5. **Masking.** All eight data masks are tried; each candidate is scored with the specification's four penalty rules (runs, 2×2 blocks, finder-like patterns, dark-ratio balance) and the lowest-penalty mask wins. Its 15-bit BCH format information is written in both standard positions.

6. **Rendering.** The final module matrix is painted to a canvas (with the mandatory 4-module quiet zone) for the PNG download, and serialised as an SVG path for the vector download.

The specification tables used — error-correction block structures and alignment positions for versions 1–20 at all four levels — were transcribed programmatically from a reference encoder's data tables rather than typed by hand, eliminating transcription errors; correctness was then proven by decoding (Section 10).

8 · Architecture & sandbox compliance

One HTML file (41,603 bytes) in three layers: presentation (inline CSS, responsive grid, studio palette — emerald `#103E32`, lime `#DCF5B5`, paper `#F7F8F2`), interface (type forms, live preview, downloads), and a **pure core** with no DOM access — payload builders, validators and the complete QR encoder (`buildPayload`, `qrEncode`) — which is unit-tested directly (Section 10).

Sandbox rule	How the app complies
No API keys or private data	None exist anywhere in the file; there is no server component at all.
No localStorage / cookies	Never used; state lives in ordinary variables for the session only.
No service workers	None registered; works from `file://` or any static host.
No browser permissions	No camera, microphone, notification, geolocation or clipboard APIs. Notably, the app *creates* codes — it never asks to scan, so it never needs the camera.
No parent-window access	`window.parent` and `postMessage` are never used; behaviour is identical standalone or embedded.
No network calls	Zero external assets, zero fetches. Downloads use in-memory object URLs, revoked when regenerated.

9 · Deployment

9.1 Asif SaaS Studio (Wix Headless + CMS)

1. Wix dashboard → CMS → **SaasProducts** → add a new item.
2. Title: *QR Code Generator — Links, Wi-Fi, Contacts & More*; Slug: `qr-code-generator`; Category: *Design*; Version: *1.0*; Demo: Yes; Published: Yes.
3. Paste the complete contents of `index.html` into the **appHtml** field.
4. Open `app.html?slug=qr-code-generator` on the live site and run the smoke test in Section 10.3.

9.2 Any other host

Upload `index.html` to any static host or open it from disk. No build step, no configuration, no environment variables.

10 · Testing

10.1 Automated tests — all passed

Area	What was verified	Result
Payload builders (31 tests)	URL normalisation and rejection; Wi-Fi payload construction and escaping incl. no-password and hidden variants; vCard structure and phone cleaning; mailto percent-encoding; SMSTO / tel / geo payloads; every documented validation error; over-capacity handling; SVG output.	31/31
Scannability (15 codes)	Matrices from the shipped encoder — versions 1, 2, 3, 6, 10, 13, 17 and 20 at EC levels L/M/Q/H, including Wi-Fi, vCard, geo, mailto and multi-byte UTF-8 (Arabic + emoji) — rendered to images and decoded with **ZXing**, the industry-standard barcode engine.	15/15 decoded exactly
Version selection (60 cases)	The chosen version agreed with the segno reference encoder in every comparison case.	60/60

A note on method: segno's own matrices differ from this encoder's by one padding codeword in byte mode — investigation traced it to segno's `write_padding_bits`, which appends a full zero codeword when the bit stream is already byte-aligned, contrary to ISO 18004 §7.4.10. This encoder follows the specification (as do ZXing and the Nayuki reference implementation); the ZXing decode results above — reading the finished symbols back with an independent scanner — are the ground truth that the codes are correct.

10.2 Manual checklist

- Open `index.html` in Chrome/Edge — loads cleanly, no console errors.
- Text: generate and scan — the phone shows the exact text.
- Website with a bare domain — preview shows `https://` added; scan opens the site.
- Wi-Fi — scan joins the network (or offers to) with the right password.
- Contact — scan offers to save the contact with every filled field.
- Clear a required field → the error appears under that field, nothing generated.
- Latitude 91 and a malformed email → range/format errors shown.
- Long text at level H → the capacity message appears; level L generates it.
- PNG at 2048 and SVG downloads save, open, and scan.
- Narrow window (~380 px) → layout stacks; all controls reachable.

10.3 Live smoke test (after CMS publishing)

On the live page, create one Website code and one Wi-Fi code inside the embedded app, download the PNG, and scan it with a phone — confirming the tool works end-to-end within the site's iframe.

11 · Troubleshooting & FAQ

Question	Answer
The code won't scan	Print/display it larger, keep the white border (quiet zone) intact, avoid glare, and test at the real distance. Very dense codes (long content at H) need bigger print. Never crop the edges.
Wi-Fi code doesn't join	Check the SSID's exact spelling and capitals, the security type, and that the password field matches the current router password. Some older Android phones struggle with hidden-network codes.
Contact saves without some fields	Only filled fields are encoded — empty fields are left out by design. Fill them and regenerate.
“Content too long” message	Shorten the content or lower the error-correction level (H → M → L). A contact card with a long note is the usual culprit.
Can I put my logo in the middle?	Not inside this app — but a code made at level H tolerates roughly 30% damage, so a small centred logo added afterwards in an image editor usually still scans. Always test the final artwork.
My link changed — can I edit the code?	No. A printed QR code is fixed forever; the content is the pattern. Generate a new code for the new link (this permanence is exactly why direct encoding beats redirect services).
Is anything uploaded or tracked?	No. The code is generated on your device; the page makes no network requests, and there are no scan analytics — a plain QR code cannot report back when scanned.

12 · Limitations

- Versions 1–20 only: a single code holds up to ≈858 bytes at level L (≈382 at H) — far beyond links, credentials and contact cards, but long documents must be shortened or linked instead.
- Byte mode only: digits-only or uppercase-only content could theoretically fit smaller symbols with numeric/alphanumeric modes; byte mode was chosen because it handles every language and symbol set uniformly.
- Black-on-white output only — inverted or coloured codes scan unreliably and are not offered.
- No dynamic/editable codes and no scan statistics (both would require a redirect server, deliberately out of scope).
- No bulk/batch generation in v1.0.

13 · Version history & credits

Version	Date	Notes
1.0	20 Oct 2026	First release: 8 content types with user-chosen fields; hand-written ISO 18004 encoder (versions 1–20, EC levels L/M/Q/H); PNG + SVG downloads; full validation and live payload preview. Verified by 31 payload tests and 15/15 independent ZXing decodes.

Designed and built for **Asif SaaS Studio**.

Creator: Dr. Asif Ahmad

The application contains no third-party code, libraries or assets; the QR specification tables were transcribed programmatically from reference data and verified by decoding.

Appendix A · The pure core (verbatim)

These are the exact functions tested in Section 10 — specification tables, Galois-field and Reed–Solomon maths, the encoder, payload builders and validators — reproduced from the shipped `index.html`. The complete file, interface included, ships as `index.html` with the product.

```
/* QR spec tables for versions 1–20 (verified against a reference encoder).
   ec[v] = [L, M, Q, H] entries of [ecCodewordsPerBlock, [[blocks, dataCwPerBlock], ...]] */
var QR_TABLES = {"ec":{"1":[[7,[[1,19]]],[10,[[1,16]]],[13,[[1,13]]],[17,[[1,9]]]],"2":[[10,[[1,34]]],[16,
[[1,28]]],[22,[[1,22]]],[28,[[1,16]]]],"3":[[15,[[1,55]]],[26,[[1,44]]],[18,[[2,17]]],[22,[[2,13]]]],"4":[[20,
[[1,80]]],[18,[[2,32]]],[26,[[2,24]]],[16,[[4,9]]]],"5":[[26,[[1,108]]],[24,[[2,43]]],[18,[[2,15],[2,16]]],
[22,[[2,11],[2,12]]]],"6":[[18,[[2,68]]],[16,[[4,27]]],[24,[[4,19]]],[28,[[4,15]]]],"7":[[20,[[2,78]]],[18,
[[4,31]]],[18,[[2,14],[4,15]]],[26,[[4,13],[1,14]]]],"8":[[24,[[2,97]]],[22,[[2,38],[2,39]]],[22,[[4,18],
[2,19]]],[26,[[4,14],[2,15]]]],"9":[[30,[[2,116]]],[22,[[3,36],[2,37]]],[20,[[4,16],[4,17]]],[24,[[4,12],
[4,13]]]],"10":[[18,[[2,68],[2,69]]],[26,[[4,43],[1,44]]],[24,[[6,19],[2,20]]],[28,[[6,15],[2,16]]]],"11":
[[20,[[4,81]]],[30,[[1,50],[4,51]]],[28,[[4,22],[4,23]]],[24,[[3,12],[8,13]]]],"12":[[24,[[2,92],[2,93]]],[22,
[[6,36],[2,37]]],[26,[[4,20],[6,21]]],[28,[[7,14],[4,15]]]],"13":[[26,[[4,107]]],[22,[[8,37],[1,38]]],[24,
[[8,20],[4,21]]],[22,[[12,11],[4,12]]]],"14":[[30,[[3,115],[1,116]]],[24,[[4,40],[5,41]]],[20,[[11,16],
[5,17]]],[24,[[11,12],[5,13]]]],"15":[[22,[[5,87],[1,88]]],[24,[[5,41],[5,42]]],[30,[[5,24],[7,25]]],[24,
[[11,12],[7,13]]]],"16":[[24,[[5,98],[1,99]]],[28,[[7,45],[3,46]]],[24,[[15,19],[2,20]]],[30,[[3,15],
[13,16]]]],"17":[[28,[[1,107],[5,108]]],[28,[[10,46],[1,47]]],[28,[[1,22],[15,23]]],[28,[[2,14],
[17,15]]]],"18":[[30,[[5,120],[1,121]]],[26,[[9,43],[4,44]]],[28,[[17,22],[1,23]]],[28,[[2,14],
[19,15]]]],"19":[[28,[[3,113],[4,114]]],[26,[[3,44],[11,45]]],[26,[[17,21],[4,22]]],[26,[[9,13],
[16,14]]]],"20":[[28,[[3,107],[5,108]]],[26,[[3,41],[13,42]]],[30,[[15,24],[5,25]]],[28,[[15,15],
[10,16]]]]},"align":{"1":[],"2":[6,18],"3":[6,22],"4":[6,26],"5":[6,30],"6":[6,34],"7":[6,22,38],"8":
[6,24,42],"9":[6,26,46],"10":[6,28,50],"11":[6,30,54],"12":[6,32,58],"13":[6,34,62],"14":[6,26,46,66],"15":
[6,26,48,70],"16":[6,26,50,74],"17":[6,30,54,78],"18":[6,30,56,82],"19":[6,30,58,86],"20":
[6,34,62,90]},"dataCw":{"1":{"L":19,"M":16,"Q":13,"H":9},"2":{"L":34,"M":28,"Q":22,"H":16},"3":{"L":55,"M":
44,"Q":34,"H":26},"4":{"L":80,"M":64,"Q":48,"H":36},"5":{"L":108,"M":86,"Q":62,"H":46},"6":{"L":136,"M":
108,"Q":76,"H":60},"7":{"L":156,"M":124,"Q":88,"H":66},"8":{"L":194,"M":154,"Q":110,"H":86},"9":{"L":232,"M":
182,"Q":132,"H":100},"10":{"L":274,"M":216,"Q":154,"H":122},"11":{"L":324,"M":254,"Q":180,"H":140},"12":{"L":
370,"M":290,"Q":206,"H":158},"13":{"L":428,"M":334,"Q":244,"H":180},"14":{"L":461,"M":365,"Q":261,"H":
197},"15":{"L":523,"M":415,"Q":295,"H":223},"16":{"L":589,"M":453,"Q":325,"H":253},"17":{"L":647,"M":507,"Q":
367,"H":283},"18":{"L":721,"M":563,"Q":397,"H":313},"19":{"L":795,"M":627,"Q":445,"H":341},"20":{"L":861,"M":
669,"Q":485,"H":385}}};
var QR_EC_INDEX = { L: 0, M: 1, Q: 2, H: 3 };
var QR_EC_BITS = { L: 1, M: 0, Q: 3, H: 2 };
var QR_REMAINDER = [0,7,7,7,7,7,0,0,0,0,0,0,0,3,3,3,3,3,3]; // index = version
var QR_MAX_VERSION = 20;

/* ---- Galois field GF(256), primitive polynomial 0x11D ---- */
var GF_EXP = [], GF_LOG = [];
(function(){
  var x = 1;
  for (var i = 0; i < 255; i++){
    GF_EXP[i] = x; GF_LOG[x] = i;
    x <<= 1; if (x & 0x100) x ^= 0x11D;
  }
  for (var j = 255; j < 512; j++) GF_EXP[j] = GF_EXP[j - 255];
})();
function gfMul(a, b){ return (a && b) ? GF_EXP[GF_LOG[a] + GF_LOG[b]] : 0; }

/* Reed–Solomon divisor (generator) for a given EC length */
function rsDivisor(degree){
  var result = []; for (var i = 0; i < degree; i++) result.push(0);
  result[degree - 1] = 1;
  var root = 1;
  for (var d = 0; d < degree; d++){
    for (var j = 0; j < result.length; j++){
      result[j] = gfMul(result[j], root);
      if (j + 1 < result.length) result[j] ^= result[j + 1];
    }
    root = gfMul(root, 2);
  }
  return result;
}
function rsRemainder(data, divisor){
  var result = []; for (var i = 0; i < divisor.length; i++) result.push(0);
  for (var k = 0; k < data.length; k++){
    var factor = data[k] ^ result.shift();
    result.push(0);
    for (var j = 0; j < divisor.length; j++) result[j] ^= gfMul(divisor[j], factor);
```

```
  }
  return result;
}

/* Total data codewords of a version at an EC level */
function qrDataCodewords(version, level){
  var entry = QR_TABLES.ec[version][QR_EC_INDEX[level]];
  var total = 0;
  entry[1].forEach(function(g){ total += g[0] * g[1]; });
  return total;
}
/* Max UTF-8 bytes that fit at the largest supported version */
function qrMaxBytes(level){
  var dcw = qrDataCodewords(QR_MAX_VERSION, level);
  return Math.floor((dcw * 8 - 20) / 8); // 4 mode bits + 16 length bits at v>=10
}

/* Build the padded data codewords for the smallest fitting version; null if too long */
function qrDataEncode(bytes, level){
  for (var v = 1; v <= QR_MAX_VERSION; v++){
    var dcw = qrDataCodewords(v, level);
    var capBits = dcw * 8;
    var lenBits = (v <= 9) ? 8 : 16;
    if (4 + lenBits + bytes.length * 8 > capBits) continue;
    var bits = [];
    function pushBits(val, n){ for (var i = n - 1; i >= 0; i--) bits.push((val >>> i) & 1); }
    pushBits(4, 4); pushBits(bytes.length, lenBits);
    bytes.forEach(function(b){ pushBits(b, 8); });
    var term = Math.min(4, capBits - bits.length);
    for (var t = 0; t < term; t++) bits.push(0);
    while (bits.length % 8) bits.push(0);
    var out = [];
    for (var i = 0; i < bits.length; i += 8){
      var byte = 0; for (var j = 0; j < 8; j++) byte = (byte << 1) | bits[i + j];
      out.push(byte);
    }
    var pads = [0xEC, 0x11], p = 0;
    while (out.length < dcw) out.push(pads[p++ % 2]);
    return { version: v, data: out };
  }
  return null;
}

/* Split into blocks, compute EC, interleave → final codeword sequence */
function qrInterleave(data, version, level){
  var entry = QR_TABLES.ec[version][QR_EC_INDEX[level]];
  var ecLen = entry[0], groups = entry[1];
  var blocks = [], ecBlocks = [], idx = 0, divisor = rsDivisor(ecLen);
  groups.forEach(function(g){
    for (var b = 0; b < g[0]; b++){
      var block = data.slice(idx, idx + g[1]); idx += g[1];
      blocks.push(block); ecBlocks.push(rsRemainder(block, divisor));
    }
  });
  var out = [], maxLen = 0;
  blocks.forEach(function(b){ if (b.length > maxLen) maxLen = b.length; });
  for (var col = 0; col < maxLen; col++)
    blocks.forEach(function(b){ if (col < b.length) out.push(b[col]); });
  for (var e = 0; e < ecLen; e++)
    ecBlocks.forEach(function(b){ out.push(b[e]); });
  return out;
}

/* ---- Matrix construction ---- */
function qrBlankMatrix(size){
  var m = [], r = [];
  for (var i = 0; i < size; i++){ m.push(new Array(size).fill(0)); r.push(new Array(size).fill(false)); }
  return { m: m, r: r };
}
function qrSet(ctx, x, y, val){ // x = column, y = row
  if (x >= 0 && x < ctx.m.length && y >= 0 && y < ctx.m.length){ ctx.m[y][x] = val; ctx.r[y][x] = true; }
}
function qrDrawFinders(ctx){
  var size = ctx.m.length;
  function finder(cx, cy){
    for (var dy = -4; dy <= 4; dy++) for (var dx = -4; dx <= 4; dx++){
```

```
      var xx = cx + dx, yy = cy + dy;
      var dist = Math.max(Math.abs(dx), Math.abs(dy));
      qrSet(ctx, xx, yy, (dist !== 2 && dist !== 4) ? 1 : 0);
    }
  }
  finder(3, 3); finder(size - 4, 3); finder(3, size - 4);
}
function qrDrawAlignment(ctx, version){
  var pos = QR_TABLES.align[version] || [];
  var size = ctx.m.length;
  for (var i = 0; i < pos.length; i++) for (var j = 0; j < pos.length; j++){
    var cx = pos[i], cy = pos[j];
    // skip positions overlapping the finder patterns
    if ((cx <= 8 && cy <= 8) || (cx >= size - 9 && cy <= 8) || (cx <= 8 && cy >= size - 9)) continue;
    for (var dy = -2; dy <= 2; dy++) for (var dx = -2; dx <= 2; dx++){
      var dist = Math.max(Math.abs(dx), Math.abs(dy));
      qrSet(ctx, cx + dx, cy + dy, dist !== 1 ? 1 : 0);
    }
  }
}
function qrFormatBits(level, mask){
  var data = (QR_EC_BITS[level] << 3) | mask;
  var v = data << 10;
  for (var i = 14; i >= 10; i--) if ((v >>> i) & 1) v ^= 0x537 << (i - 10);
  return ((data << 10) | (v & 0x3FF)) ^ 0x5412;
}
function qrVersionBits(version){
  var rem = version;
  for (var i = 0; i < 12; i++) rem = (rem << 1) ^ (((rem >>> 11) & 1) ? 0x1F25 : 0);
  return (version << 12) | (rem & 0xFFF);
}
function qrDrawFormatAndVersion(ctx, version, level, mask){
  var size = ctx.m.length;
  var bits = qrFormatBits(level, mask);
  function bit(i){ return (bits >>> i) & 1; }
  var i;
  for (i = 0; i <= 6; i++) qrSet(ctx, 8, i, bit(i));
  qrSet(ctx, 8, 7, bit(6)); qrSet(ctx, 8, 8, bit(7)); qrSet(ctx, 7, 8, bit(8));
  for (i = 9; i < 15; i++) qrSet(ctx, 14 - i, 8, bit(i));
  for (i = 0; i < 8; i++) qrSet(ctx, size - 1 - i, 8, bit(i));
  for (i = 8; i < 15; i++) qrSet(ctx, 8, size - 15 + i, bit(i));
  qrSet(ctx, 8, size - 8, 1); // dark module
  if (version >= 7){
    var vb = qrVersionBits(version);
    for (i = 0; i < 18; i++){
      var b = (vb >>> i) & 1;
      qrSet(ctx, size - 11 + (i % 3), Math.floor(i / 3), b);
      qrSet(ctx, Math.floor(i / 3), size - 11 + (i % 3), b);
    }
  }
}
function qrMaskCondition(mask, r, c){ // r = row, c = column
  switch (mask){
    case 0: return (r + c) % 2 === 0;
    case 1: return r % 2 === 0;
    case 2: return c % 3 === 0;
    case 3: return (r + c) % 3 === 0;
    case 4: return (Math.floor(r / 2) + Math.floor(c / 3)) % 2 === 0;
    case 5: return (r * c) % 2 + (r * c) % 3 === 0;
    case 6: return ((r * c) % 2 + (r * c) % 3) % 2 === 0;
    case 7: return ((r + c) % 2 + (r * c) % 3) % 2 === 0;
  }
  return false;
}
function qrPenalty(m){
  var size = m.length, score = 0, r, c, i, j;
  // Rule 1: runs of >= 5
  function runs(line){
    var s = 0, run = 1;
    for (var i = 1; i <= line.length; i++){
      if (i < line.length && line[i] === line[i - 1]) run++;
      else { if (run >= 5) s += run - 2; run = 1; }
    }
    return s;
  }
  for (r = 0; r < size; r++){ score += runs(m[r]); var col = []; for (c = 0; c < size; c++) col.push(m[c][r]);
```

```
score += runs(col); }
  // Rule 2: 2x2 blocks
  for (r = 0; r < size - 1; r++) for (c = 0; c < size - 1; c++)
    if (m[r][c] === m[r][c + 1] && m[r][c] === m[r + 1][c] && m[r][c] === m[r + 1][c + 1]) score += 3;
  // Rule 3: finder-like patterns with a 4-module quiet zone on either side
  var pat1 = [1,0,1,1,1,0,1,0,0,0,0], pat2 = [0,0,0,0,1,0,1,1,1,0,1];
  function countPat(line){
    var s = 0;
    for (var i = 0; i + 11 <= line.length; i++){
      var w = line.slice(i, i + 11);
      var eq = function(p){ for (var j = 0; j < 11; j++) if (w[j] !== p[j]) return false; return true; };
      if (eq(pat1) || eq(pat2)) s += 40;
    }
    return s;
  }
  for (r = 0; r < size; r++){ score += countPat(m[r]); var col2 = []; for (c = 0; c < size; c++)
col2.push(m[c][r]); score += countPat(col2); }
  // Rule 4: dark-module proportion
  var dark = 0;
  for (r = 0; r < size; r++) for (c = 0; c < size; c++) dark += m[r][c];
  var total = size * size;
  var k = Math.ceil(Math.abs(dark * 20 - total * 10) / total) - 1;
  if (k > 0) score += k * 10;
  return score;
}

/* Encode text into a QR matrix. forceMask: 0–7 or -1/undefined for automatic.
   Returns { version, size, matrix } or null when the text is too long. */
function qrEncode(text, level, forceMask){
  var bytes = Array.prototype.slice.call(new TextEncoder().encode(text));
  var enc = qrDataEncode(bytes, level);
  if (!enc) return null;
  var codewords = qrInterleave(enc.data, enc.version, level);
  var bits = [];
  codewords.forEach(function(b){ for (var i = 7; i >= 0; i--) bits.push((b >>> i) & 1); });
  for (var rb = 0; rb < QR_REMAINDER[enc.version]; rb++) bits.push(0);

  var size = 17 + 4 * enc.version;
  var best = null, bestPenalty = Infinity;
  var masks = (forceMask !== undefined && forceMask >= 0) ? [forceMask] : [0,1,2,3,4,5,6,7];
  masks.forEach(function(mask){
    var ctx = qrBlankMatrix(size);
    qrDrawFinders(ctx);
    // timing patterns
    for (var i = 0; i < size; i++){
      if (!ctx.r[6][i]) qrSet(ctx, i, 6, i % 2 === 0 ? 1 : 0);
      if (!ctx.r[i][6]) qrSet(ctx, 6, i, i % 2 === 0 ? 1 : 0);
    }
    qrDrawAlignment(ctx, enc.version);
    qrDrawFormatAndVersion(ctx, enc.version, level, mask);
    // data placement, zigzag from bottom-right, skipping column 6
    var bitIdx = 0;
    for (var right = size - 1; right >= 1; right -= 2){
      if (right === 6) right = 5;
      for (var vert = 0; vert < size; vert++){
        for (var j = 0; j < 2; j++){
          var x = right - j;
          var upward = ((right + 1) & 2) === 0;
          var y = upward ? size - 1 - vert : vert;
          if (!ctx.r[y][x]){
            var bit = bitIdx < bits.length ? bits[bitIdx] : 0; bitIdx++;
            if (qrMaskCondition(mask, y, x)) bit ^= 1;
            ctx.m[y][x] = bit;
          }
        }
      }
    }
    if (masks.length === 1){ best = ctx.m; }
    else {
      var pen = qrPenalty(ctx.m);
      if (pen < bestPenalty){ bestPenalty = pen; best = ctx.m; }
    }
  });
  return { version: enc.version, size: size, matrix: best };
}
```

```
/* Render a matrix as a standalone SVG string (4-module quiet zone) */
function qrSvgString(matrix, modulePx){
  var size = matrix.length, total = size + 8;
  var px = modulePx || 8;
  var d = "";
  for (var r = 0; r < size; r++) for (var c = 0; c < size; c++)
    if (matrix[r][c]) d += "M" + (c + 4) + " " + (r + 4) + "h1v1h-1Z";
  return "<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"" + (total * px) +
    "\" height=\"" + (total * px) + "\" viewBox=\"0 0 " + total + " " + total +
    "\"><rect width=\"100%\" height=\"100%\" fill=\"#ffffff\"/><path d=\"" + d + "\" fill=\"#000000\"/></
svg>";
}

/* ================= PAYLOAD BUILDERS + VALIDATION =================
   Each returns { payload } or { errors: { fieldKey: message } } */
function escMeCard(s){ return String(s).replace(/\\/g, "\\\\").replace(/;/g, "\\;").replace(/,/g, "\
\,").replace(/"/g, "\\\"").replace(/:/g, "\\:").replace(/\r?\n/g, "\\n"); }
function escVCard(s){ return String(s).replace(/\\/g, "\\\\").replace(/\r?\n/g, "\\n").replace(/,/g, "\
\,").replace(/;/g, "\\;"); }
var EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
var PHONE_RE = /^\+?[0-9\s()\-]{5,24}$/;
function cleanPhone(s){ return String(s).replace(/[\s()]/g, ""); }

function buildPayload(type, f){
  var errors = {}, payload = "";
  function req(key, val, msg){ if (!String(val || "").trim()){ errors[key] = msg; return ""; } return
String(val).trim(); }
  if (type === "text"){
    payload = req("text", f.text, "Enter some text for the QR code.");
    if (!String(f.text || "").trim()) payload = "";
    else payload = String(f.text);
  }
  else if (type === "url"){
    var u = req("url", f.url, "Enter a website address.");
    if (u){
      if (!/^[a-zA-Z][a-zA-Z0-9+.-]*:\/\//.test(u)) u = "https://" + u;
      try { var parsed = new URL(u); if (parsed.protocol !== "http:" && parsed.protocol !== "https:") throw
new Error("scheme"); payload = u; }
      catch(e){ errors.url = "That does not look like a valid website address."; }
    }
  }
  else if (type === "wifi"){
    var ssid = req("wifiSsid", f.wifiSsid, "Enter the network name (SSID).");
    var enc = f.wifiEnc || "WPA";
    var pass = String(f.wifiPass || "");
    if (enc !== "nopass" && !pass) errors.wifiPass = "Enter the Wi-Fi password, or choose “No password”.";
    if (ssid && !errors.wifiPass){
      payload = "WIFI:T:" + enc + ";S:" + escMeCard(ssid) + ";";
      if (enc !== "nopass") payload += "P:" + escMeCard(pass) + ";";
      if (f.wifiHidden) payload += "H:true;";
      payload += ";";
    }
  }
  else if (type === "vcard"){
    var name = req("vcName", f.vcName, "Enter at least a full name for the contact card.");
    if (f.vcPhone && !PHONE_RE.test(String(f.vcPhone))) errors.vcPhone = "That phone number looks invalid.";
    if (f.vcEmail && !EMAIL_RE.test(String(f.vcEmail).trim())) errors.vcEmail = "That email address looks
invalid.";
    if (name && !errors.vcPhone && !errors.vcEmail){
      var parts = name.split(/\s+/);
      var first = parts[0], last = parts.length > 1 ? parts[parts.length - 1] : "";
      var lines = ["BEGIN:VCARD", "VERSION:3.0",
        "N:" + escVCard(last) + ";" + escVCard(first) + ";;;",
        "FN:" + escVCard(name)];
      if (f.vcOrg) lines.push("ORG:" + escVCard(f.vcOrg));
      if (f.vcTitle) lines.push("TITLE:" + escVCard(f.vcTitle));
      if (f.vcPhone) lines.push("TEL;TYPE=CELL:" + cleanPhone(f.vcPhone));
      if (f.vcEmail) lines.push("EMAIL:" + String(f.vcEmail).trim());
      if (f.vcUrl) lines.push("URL:" + String(f.vcUrl).trim());
      if (f.vcStreet || f.vcCity || f.vcZip || f.vcCountry)
        lines.push("ADR:;;" + escVCard(f.vcStreet || "") + ";" + escVCard(f.vcCity || "") + ";;" +
          escVCard(f.vcZip || "") + ";" + escVCard(f.vcCountry || ""));
      if (f.vcNote) lines.push("NOTE:" + escVCard(f.vcNote));
      lines.push("END:VCARD");
      payload = lines.join("\n");
    }
```

```
  }
  else if (type === "email"){
    var to = req("emTo", f.emTo, "Enter the recipient's email address.");
    if (to && !EMAIL_RE.test(to)) errors.emTo = "That email address looks invalid.";
    if (to && !errors.emTo){
      var params = [];
      if (f.emSubject) params.push("subject=" + encodeURIComponent(f.emSubject));
      if (f.emBody) params.push("body=" + encodeURIComponent(f.emBody));
      payload = "mailto:" + to + (params.length ? "?" + params.join("&") : "");
    }
  }
  else if (type === "sms"){
    var num = req("smsNumber", f.smsNumber, "Enter a phone number.");
    if (num && !PHONE_RE.test(num)) errors.smsNumber = "That phone number looks invalid.";
    if (num && !errors.smsNumber){
      payload = "SMSTO:" + cleanPhone(num);
      if (f.smsMsg) payload += ":" + f.smsMsg;
    }
  }
  else if (type === "phone"){
    var ph = req("phNumber", f.phNumber, "Enter a phone number.");
    if (ph && !PHONE_RE.test(ph)) errors.phNumber = "That phone number looks invalid.";
    if (ph && !errors.phNumber) payload = "tel:" + cleanPhone(ph);
  }
  else if (type === "geo"){
    var latS = req("geoLat", f.geoLat, "Enter the latitude."), lngS = req("geoLng", f.geoLng, "Enter the
longitude.");
    var lat = Number(latS), lng = Number(lngS);
    if (latS && (isNaN(lat) || lat < -90 || lat > 90)) errors.geoLat = "Latitude must be a number between −90
and 90.";
    if (lngS && (isNaN(lng) || lng < -180 || lng > 180)) errors.geoLng = "Longitude must be a number between
−180 and 180.";
    if (latS && lngS && !errors.geoLat && !errors.geoLng) payload = "geo:" + lat + "," + lng;
  }
  if (Object.keys(errors).length) return { errors: errors };
  return { payload: payload };
}
```